



SI100B: Introduction to Information Science and Technology



Lecture 12

Announcement

- ▶ Homework 3
 - ▶ Instructions & problems
 - ▶ BB → 作业 → “Programming Assignment 3”
 - ▶ Submission website: OJ
 - ▶ Due: 9pm, Nov. 6



Next Week

- ▶ No class next Wed & Fri
- ▶ Class on Sunday (11/9), 6-9:30pm
- ▶ Topic: introduction to AI



Midterm Exam

- ▶ Time

- ▶ Nov. 12 (Wednesday) Class 3-4
- ▶ Nov. 14 (Friday) Class 3-4
- ▶ Nov. 14 (Friday) Class 11-12
- ▶ Different problem sets. Grades will be aligned.

- ▶ Location

- ▶ 信息学院机房 3-101、1A-109

- ▶ Your exam time, room, seat assignments

- ▶ TBA



Midterm Exam

- ▶ Format
 - ▶ Programming, submission & grading at OJ
 - ▶ Closed-book. You can bring **an A4-size cheat sheet** (you can write or print anything on it). We will provide scratch paper.
- ▶ Topics
 - ▶ Everything about Python covered in class (except “OOP” & “inheritance”)
 - ▶ Lower difficulty and simpler English than homework
- ▶ Grade
 - ▶ 35% of the total grade



Midterm Exam

- ▶ Computer system
 - ▶ Windows
 - ▶ VS Code & PyCharm
 - ▶ VS Code plugins: 简体中文, python, pylint, pylance, jupyter
 - ▶ **No internet** (except for OJ)
 - ▶ OJ account: we will give you a new account right before the exam
- ▶ We will open the computer rooms to you in week 9 (time TBA), so you can familiarize yourself with the exam computer system



Midterm Exam

▶ Rules

- ▶ Bring your **photo ID** (e.g., student card).
- ▶ Before the exam, put all the study materials and electronic devices into your bag and put your bag in the front, back, or sides of the classroom.
- ▶ We will record videos / take photos during the exam. Please discipline yourself.



HOW TO DEFINE A CLASS (RECAP)

```
class Animal(object):  
    def __init__(self, age):  
        self.age = age  
        self.name = None
```

```
myanimal = Animal(3)
```

*name is a data attribute
even though an instance
is not initialized with it as
a param*



GETTER AND SETTER METHODS

```
class Animal(object):  
    def __init__(self, age):  
        self.age = age  
        self.name = None  
    def __str__(self):  
        return "animal:" + str(self.name) + ":" + str(self.age)  
    def get_age(self):  
        return self.age  
    def get_name(self):  
        return self.name  
    def set_age(self, newage):  
        self.age = newage  
    def set_name(self, newname=""):  
        self.name = newname
```

getter

setter



AN INSTANCE and DOT NOTATION (RECAP)

```
a = Animal(3)
```

```
a.age
```

```
a.get_age()
```

- access method
- best to use getters and setters

- access data attribute
- allowed, but not recommended

- ▶ Dot notation used to access attributes (data and methods) though it is better to use **getters and setters** to access data attributes



CHANGING INTERNAL REPRESENTATION

- ▶ Author of class definition may **change data attribute**

```
class Animal(object):  
    def __init__(self, age):  
        self.years = age  
        self.name = None  
    def __str__(self):  
        return "animal:" + str(self.name) + ":" + str(self.age)  
    def get_age(self):  
        return self.years  
    def set_age(self, newage):  
        self.years = newage
```

```
a.get_age()    # works  
a.age          # error
```

*Change internal rep from
self.age = age*

*Accessing methods works
correctly, but accessing
data attributes no longer
works*

- ▶ Getters and setters should be used outside of class to access data attributes
-



PYTHON NOT GREAT AT INFORMATION HIDING

- ▶ Allows you to **access data** from outside class definition
`print(a.age)`
- ▶ Allows you to **write to data** from outside class definition
`a.age = 'infinite'`
- ▶ Allows you to **create data attributes** for an instance from outside class definition
`a.size = "tiny"`
- ▶ It's **not good style** to do any of these!
- ▶ You should only interact with objects through methods!
 - ▶ E.g., getters and setters
- ▶ Remember methods are the interface for interaction



BIG IDEA

Access data attributes

(stuff defined by `self.xxx`)

through methods – it's
better style.





INHERITANCE

HIERARCHIES

Animal



Person

Student



Cat



Rabbit



HIERARCHIES

- ▶ **Parent class**

(superclass)

- ▶ **Child class**

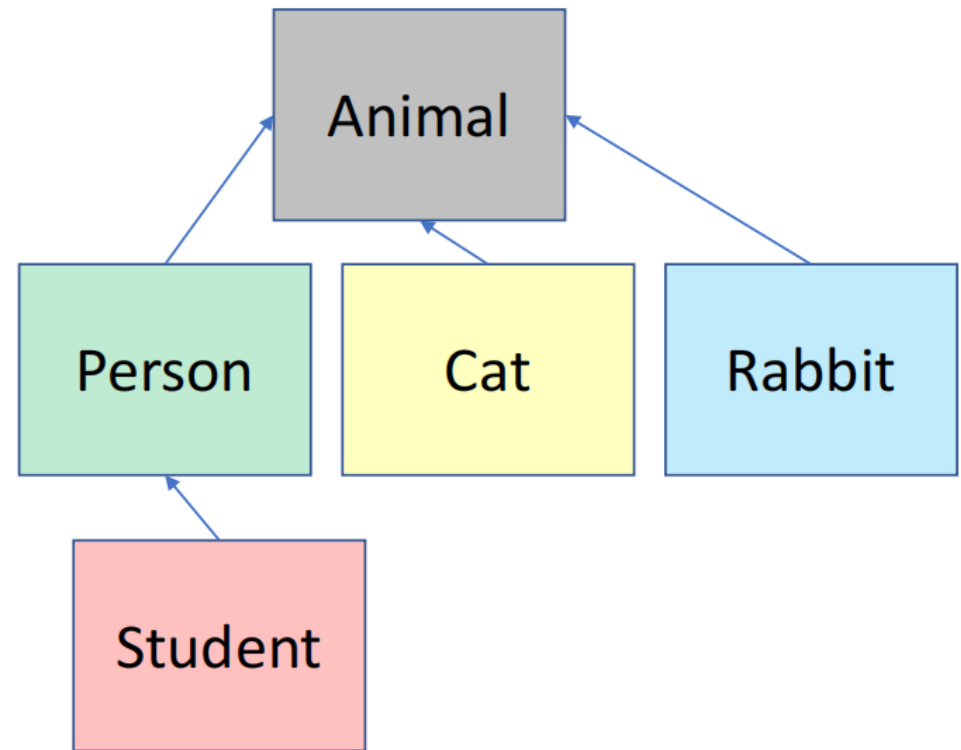
(subclass)

- ▶ **Inherits** all data and behaviors of parent class

- ▶ **Add** more **info**

- ▶ **Add** more **behavior**

- ▶ **Override** behavior



INHERITANCE:

PARENT CLASS

```
class Animal(object):  
    def __init__(self, age):  
        self.age = age  
        self.name = None  
    def get_age(self):  
        return self.age  
    def get_name(self):  
        return self.name  
    def set_age(self, newage):  
        self.age = newage  
    def set_name(self, newname=""):  
        self.name = newname  
    def __str__(self):  
        return "animal:" + str(self.name) + ":" + str(self.age)
```

- Everything is an object
- Class object
- Implements basic operations in Python, like binding variables, etc



SUBCLASS CAT

```
class Cat(Animal):  
    def speak(self):  
        print("meow")  
    def __str__(self):  
        return "cat:" + str(self.name) + ":" + str(self.age)
```

Add new functionality
via speak method

Inherits all attributes of Animal:
__init__()
age, name
get_age(), get_name()
set_age(), set_name()
__str__()

Overrides __str__()

- ▶ `__init__()` is not missing, uses the Animal version
- ▶ Add new functionality with `speak()`
 - ▶ Instance of type Cat can be called with new methods
 - ▶ Instance of type Animal throws error if called with Cat's new method



WHICH METHOD TO USE?

- ▶ Subclass can have **methods with same name** as superclass
- ▶ For an instance of a class, look for a method name in **current class definition**
- ▶ If not found, look for method name **up the hierarchy** (in parent, then grandparent, and so on)
- ▶ Use first method up the hierarchy that you found with that method name



SUBCLASS PERSON

```
class Person(Animal):
```

```
    def __init__(self, name, age):
```

```
        Animal.__init__(self, age)
```

```
        self.set_name(name)
```

```
        self.friends = []
```

```
    def get_friends(self):
```

```
        return self.friends.copy()
```

```
    def add_friend(self, fname):
```

```
        if fname not in self.friends:
```

```
            self.friends.append(fname)
```

```
    def speak(self):
```

```
        print("hello")
```

```
    def age_diff(self, other):
```

```
        diff = self.age - other.age
```

```
        print(abs(diff), "year difference")
```

```
    def __str__(self):
```

```
        return "person:" + str(self.name) + ":" + str(self.age)
```

Parent class is Animal

Call Animal constructor to run lines
of code in Animal's init
Call Animal's method
Add a new data structure

New methods

Override Animal's `__str__`
method



BIG IDEA

A subclass can
use a parent's attributes,
override a parent's attributes, or
define new attributes.

Attributes are either data or methods.



SUBCLASS STUDENT

```
import random
```

Bring in functions from
random library

```
class Student(Person):
```

```
    def __init__(self, name, age, major=None):
```

```
        Person.__init__(self, name, age)
```

```
        self.major = major
```

```
    def change_major(self, major):
```

```
        self.major = major
```

```
    def speak(self):
```

```
        r = random.random()
```

```
        if r < 0.25:
```

```
            print("i have homework")
```

```
        elif 0.25 <= r < 0.5:
```

```
            print("i need sleep")
```

```
        elif 0.5 <= r < 0.75:
```

```
            print("i should eat")
```

```
        else:
```

```
            print("i'm still zooming")
```

```
    def __str__(self):
```

```
        return "student:" + str(self.name) + ":" + str(self.age) + ":" + str(self.major)
```

Inherits Person and Animal attributes

Person __init__ takes care of all
initializations
self.major adds new data

Look up how to use random library in the python docs
random() method gives back float number in [0,1)



SUBCLASS RABBIT

- ▶ **Class variables** and their values are shared between all instances of a class

```
class Rabbit(Animal):
```

Parent class

```
    tag = 1
```

Shared class variable

```
    def __init__(self, age, parent1=None, parent2=None):
```

```
        Animal.__init__(self, age)
```

```
        self.parent1 = parent1
```

```
        self.parent2 = parent2
```

```
        self.rid = Rabbit.tag
```

```
        Rabbit.tag += 1
```

Instance variable

*Access shared class variable
Incrementing class variable changes it
for all instances that may reference it*

- ▶ tag used to give **unique id** to each new rabbit instance



RECALL THE `__init__` OF Rabbit

```
def __init__(self, age, parent1=None, parent2=None):  
    Animal.__init__(self, age)  
    self.parent1 = parent1  
    self.parent2 = parent2  
    self.rid = Rabbit.tag  
    Rabbit.tag += 1
```

Shared across all instances

Rabbit.tag

1

r1

r1 = Rabbit(8)

Age: 8
Parent1: None
Parent2: None
Rid: 1



RECALL THE `__init__` OF Rabbit

```
def __init__(self, age, parent1=None, parent2=None):  
    Animal.__init__(self, age)  
    self.parent1 = parent1  
    self.parent2 = parent2  
    self.rid = Rabbit.tag  
    Rabbit.tag += 1
```

Shared across all instances

```
r1 = Rabbit(8)
```

```
r2 = Rabbit(6)
```

Rabbit.tag

2

r1

Age: 8
Parent1: None
Parent2: None
Rid: 1

r2

Age: 6
Parent1: None
Parent2: None
Rid: 2



RECALL THE `__init__` OF Rabbit

```
def __init__(self, age, parent1=None, parent2=None):  
    Animal.__init__(self, age)  
    self.parent1 = parent1  
    self.parent2 = parent2  
    self.rid = Rabbit.tag  
    Rabbit.tag += 1
```

Shared across all instances

```
r1 = Rabbit(8)  
r2 = Rabbit(6)  
r3 = Rabbit(10)
```

Rabbit.tag

4

r1

Age: 8
Parent1: None
Parent2: None
Rid: 1

r2

Age: 6
Parent1: None
Parent2: None
Rid: 2

r3

Age: 10
Parent1: None
Parent2: None
Rid: 3



BIG IDEA

Class variables are shared between all instances.

If one instance changes it, it's changed for every instance.



WORKING WITH YOUR OWN TYPES

- ▶ Define **+ operator** between two Rabbit instances
 - ▶ Define what something like this does: $r4 = r1 + r2$ where $r1$ and $r2$ are Rabbit instances
 - ▶ $r4$ is a new Rabbit instance with age 0
 - ▶ $r4$ has $r1$ as one parent and $r2$ as the other parent

```
def __add__(self, other):  
    # returning object of same type as this class  
    return Rabbit(0, self, other)
```

recall Rabbit's `__init__(self, age, parent1=None, parent2=None)`

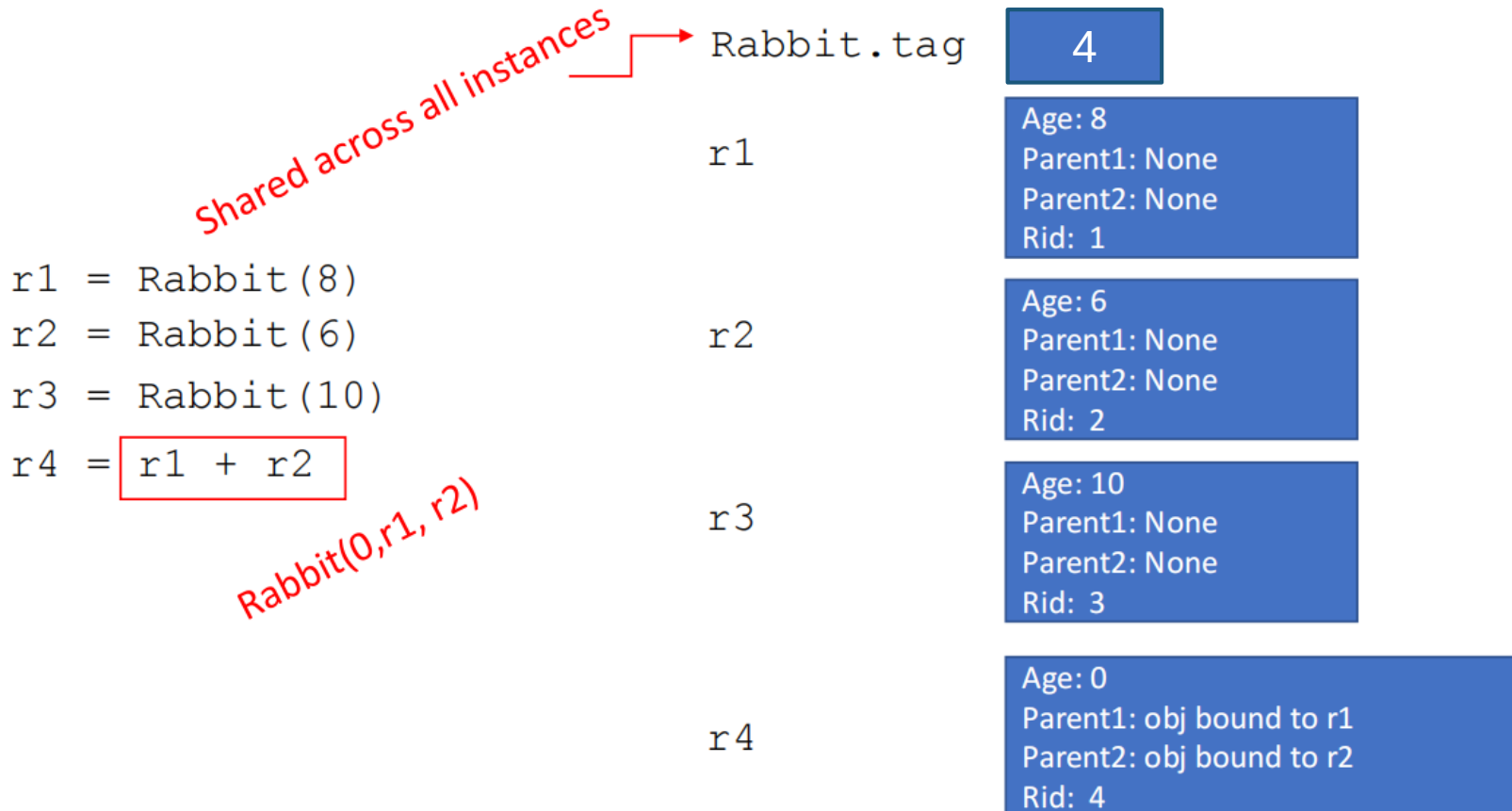


The diagram shows three red arrows pointing from the arguments of the `Rabbit(0, self, other)` call to the parameters of the `__init__` method. The first arrow points from `0` to `age`. The second arrow points from `self` to `parent1`. The third arrow points from `other` to `parent2`.



RECALL THE `__init__` OF Rabbit

```
def __init__(self, age, parent1=None, parent2=None):  
    Animal.__init__(self, age)  
    self.parent1 = parent1  
    self.parent2 = parent2  
    self.rid = Rabbit.tag  
    Rabbit.tag += 1
```



WHY USE INHERITENCE?

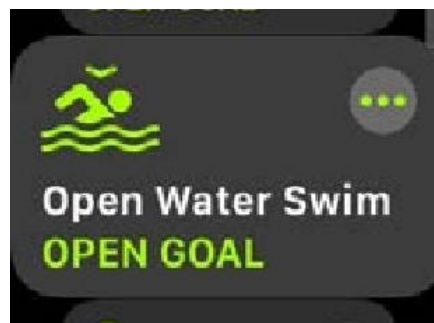
- ▶ Improve **clarity**
 - ▶ Commonalities are explicit in parent class
 - ▶ Differences are explicit in subclass
- ▶ **Reuse** code
- ▶ Enhance **modularity**



OOP EXAMPLE: FITNESS TRACKER

Workout Tracker Example

- ▶ Suppose we are writing a program to track workouts, e.g., for a smart watch



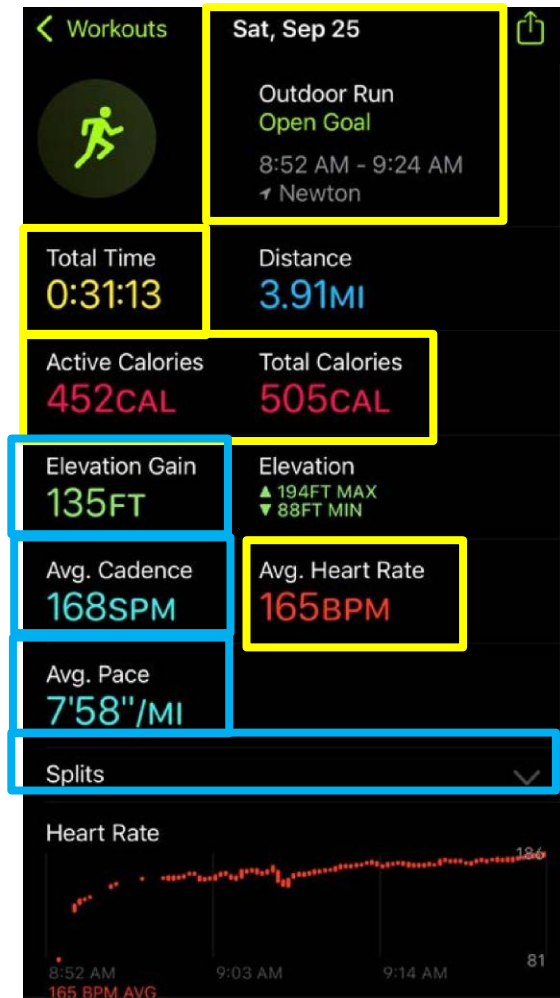
Different kinds of workouts



Fitness Tracker



Different types of workouts



Common properties:

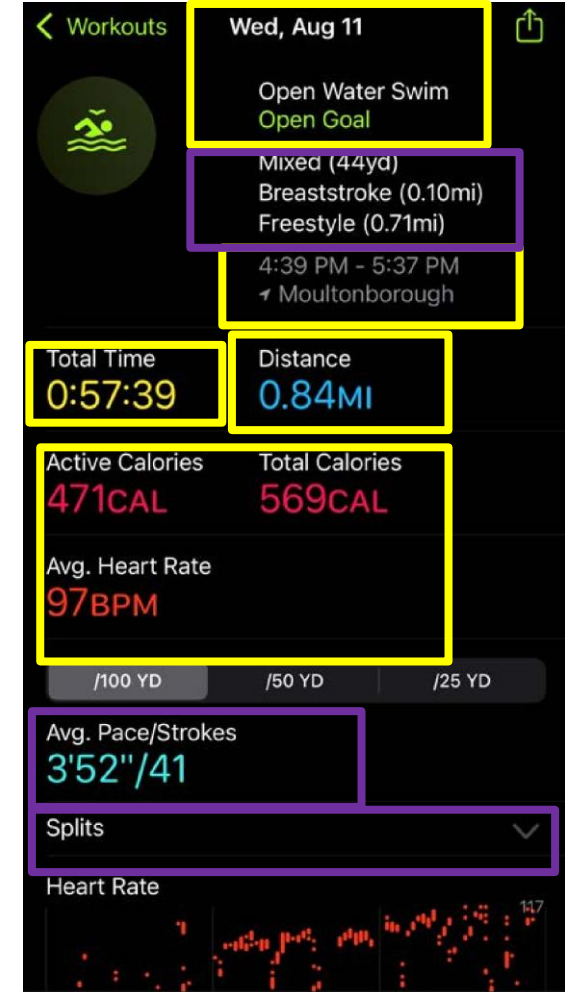
Icon	Kind
Date	Start Time
End Time	Calories
Heart Rate	Distance

Swimming Specific:

Swimming Pace
Stroke Type
100 yd Splits

Running Specific:

Cadence
Running Pace
Mile Splits
Elevation



DEFINE A SIMPLE CLASS (RECAP)

```
class Workout(object):  
    def __init__(self, start, end, calories):  
        self.start = start  
        self.end = end  
        self.calories = calories  
        self.icon = '😞'  
        self.kind = 'Workout'
```

Icon and kind are attributes even
though an instance is not
initialized with them as a param
(And python strings can contain
emojis! 🍕)

```
my_workout = Workout('9/30/2021 1:35 PM', '9/30/2021 1:57 PM', 200)
```



GETTER AND SETTER METHODS (RECAP)

```
class Workout(object):  
    def __init__(self, start, end, calories):  
        self.start = start  
        self.end = end  
        self.calories = calories  
        self.icon = 😞  
        self.kind = 'Workout'
```

getter

```
def get_calories(self):  
    return self.calories  
def get_start(self):  
    return self.start  
def get_end(self):  
    return self.end
```

setter

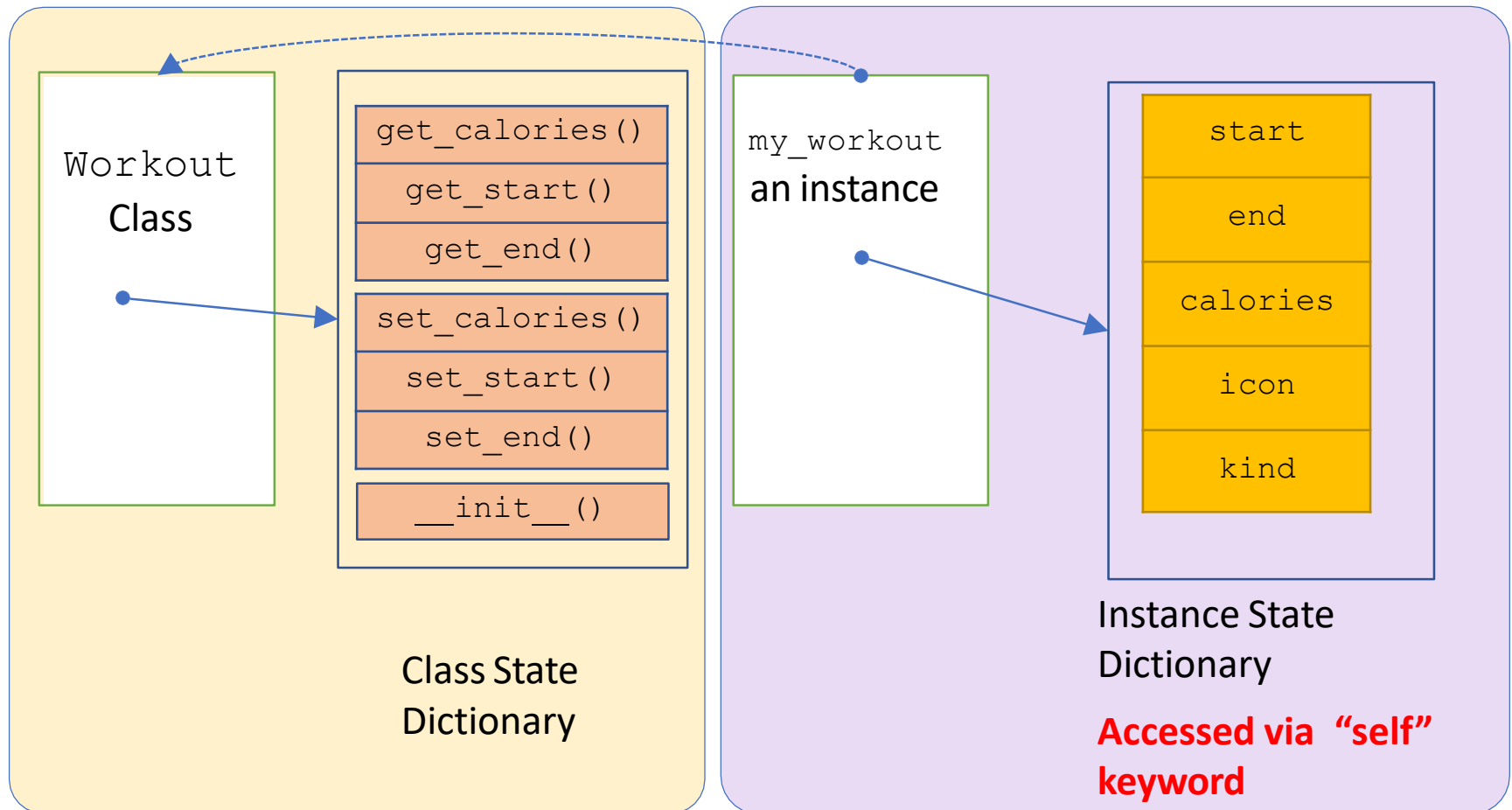
```
def set_calories(self, calories):  
    self.calories = calories  
def set_start(self, start):  
    self.start = start  
def set_end(self, end):  
    self.end = end
```

Getters and setters used outside of class to access data attributes



SELF PROVIDES ACCESS TO CLASS STATE

```
my_workout = Workout('9/30/2021 1:35 PM', '9/30/2021 1:57 PM', 200)
```



CHANGING THE CLASS IMPLEMENTATION

```
class Workout:
```

```
    cal_per_hr = 200
```

```
    def __init__(self, start, end, calories=None):
```

```
        self.start = parser.parse(start)
```

```
        self.end = parser.parse(end)
```

```
        self.calories = calories # may be None
```

```
        self.icon = '😓'
```

```
        self.kind = 'Workout'
```

```
    def get_calories(self):
```

```
        if (self.calories == None):
```

```
            return Workout.cal_per_hr * (self.end - self.start).total_seconds() / 3600
```

```
        else:
```

```
            return self.calories
```

Class variable – all instances
of Workout can read this

Defaults to None if not
passed in
self.start and self.end are objects
of type datetime, not strings

If calories was not passed in,
estimate based on elapsed time

Allowed on datetime objects

If calories was
passed in, just
use that value



ASIDE: datetime OBJECTS

OTHER PYTHON LIBRARIES

- ▶ Takes the string representing the date and time and **converts it to a datetime object**

```
from dateutil import parser
```

Brings in a bunch of
functions and classes

```
start = '9/30/2021 1:35 PM'
```

```
end = '9/30/2021 1:45 PM'
```

```
start_date = parser.parse(start)
```

```
end_date = parser.parse(end)
```

```
type(start_date)
```

Type is datetime.datetime

- ▶ Why do this? Because it **makes operations with dates easy!** The datetime object takes care of everything

```
print((end_date-start_date).total_seconds())
```

Prints 600



CLASS VARIABLES

- ▶ Associate a **class variable with all instances** of a class
- ▶ Warning: if an instance changes the class variable, it's changed for all instances

```
class Workout:
```

```
    cal_per_hr = 200
```

```
    def __init__(self, start, end, calories):
```

```
        ...
```

cal_per_hr is set to 200 for all new instances of Workout

```
print(Workout.cal_per_hr)
```

No instance required, prints 200

```
w = Workout('1/1/2021 2:34', '1/1/2021 3:35', None)
```

```
print(w.cal_per_hr)
```

Prints 200

```
Workout.cal_per_hr = 250
```

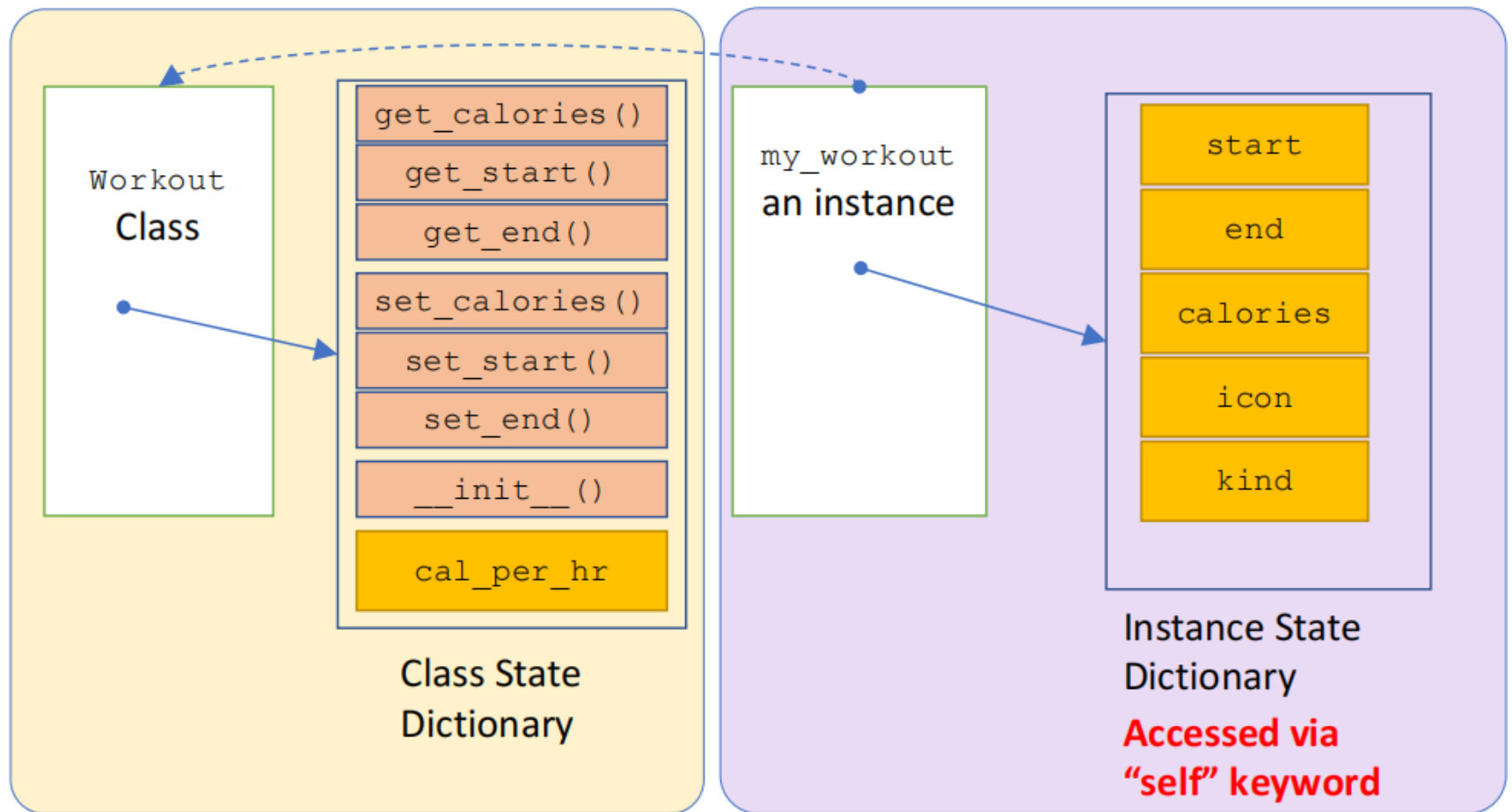
```
print(w.cal_per_hr)
```

Prints 250

Bad style to change the class variable outside the class definition. Write a method to do it!

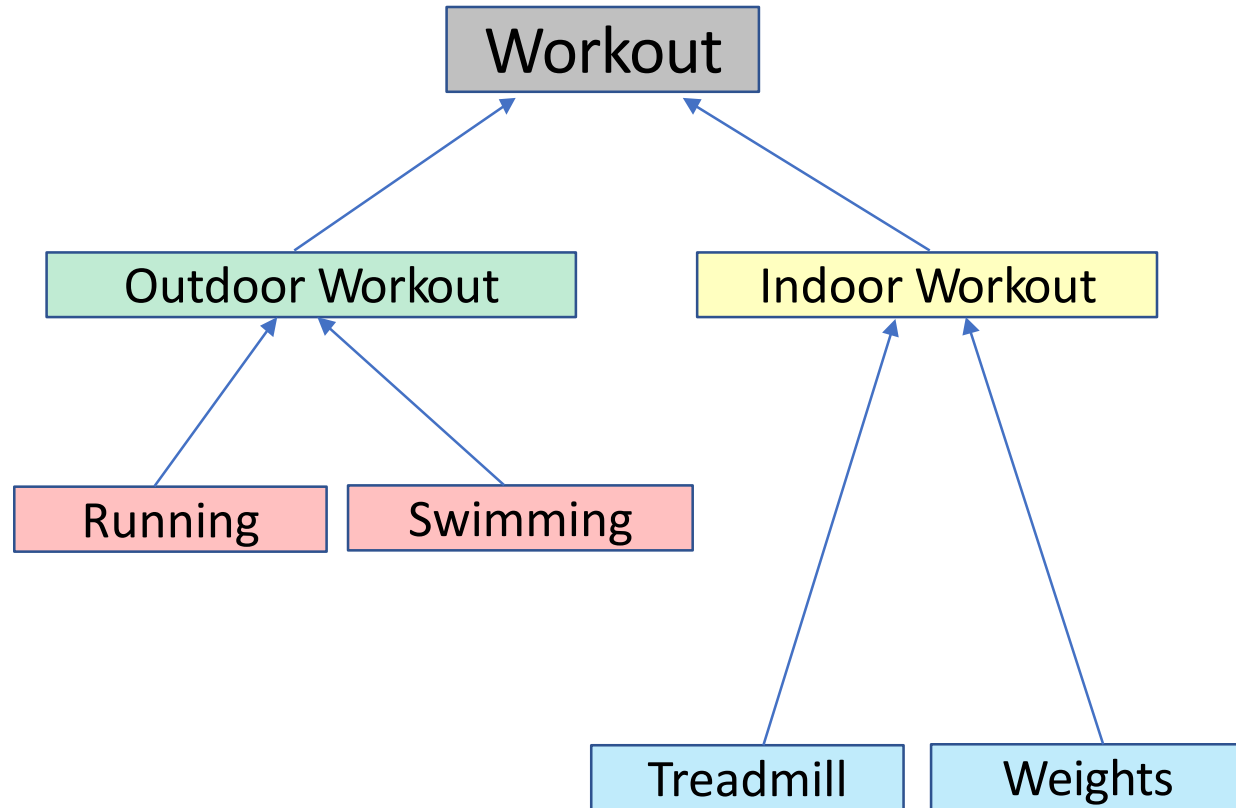


CLASS VARIABLES LIVE IN CLASS STATE DICTIONARY

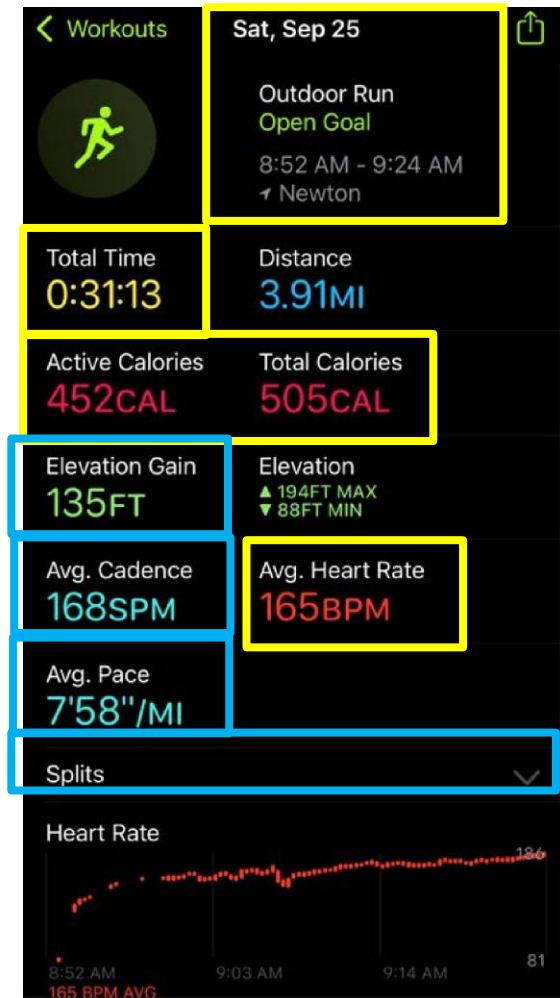


HIERARCHIES

- ▶ **Parent class**
(superclass)
- ▶ **Child class**
(subclass)
 - ▶ **Inherits** all data and behaviors of parent class
 - ▶ **Add** more **info**
 - ▶ **Add** more **behavior**
 - ▶ **Override** behavior



Fitness Tracker



Common properties:

Icon
Date
End Time
Heart Rate
Distance

Workout
"Superclass"

Swimming Specific:

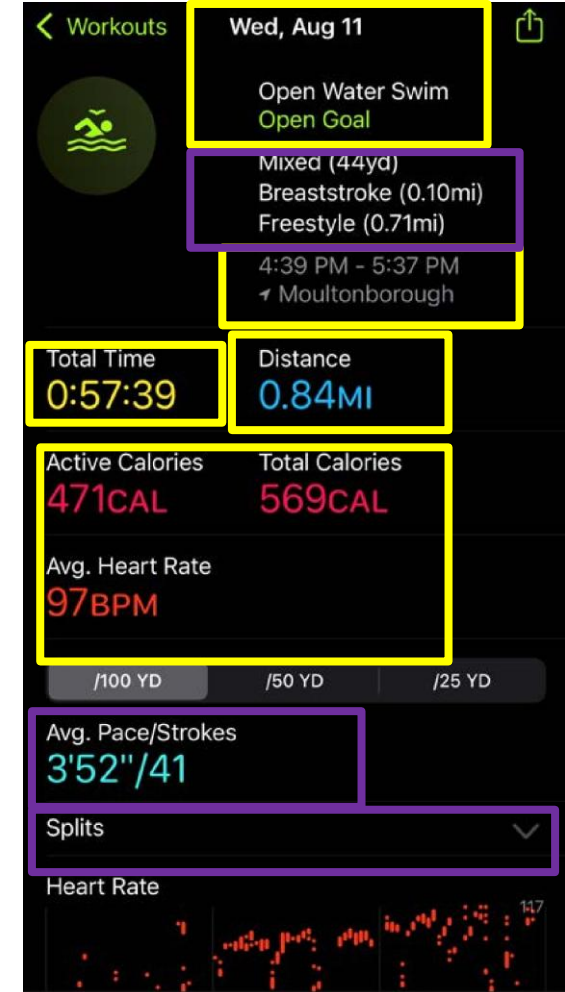
Swimming Pa
Strokes
100

Swimming
"Subclass"

Running Specific:

Cadence
Running
Mile
Eleva

Running
"Subclass"



INHERITANCE: SUBCLASS

```
class RunWorkout(Workout):
```

Parent is Workout
Inherits all attributes of Workout:
start,end,calories
get_calories(), get_start()
get_end(),... ,__str__()

```
    def __init__(self, start, end, elev=0, calories=None):
```

```
        super().__init__(start,end,calories)
```

```
        self.icon = '🏃'
```

```
        self.kind = 'Running'
```

```
        self.elev = elev
```

```
    def get_elev(self):
```

```
        return self.elev
```

```
    def set_elev(self, e):
```

```
        self.elev = e
```

Parent accessed
via super()

Add new instance
variables

Override parents
default

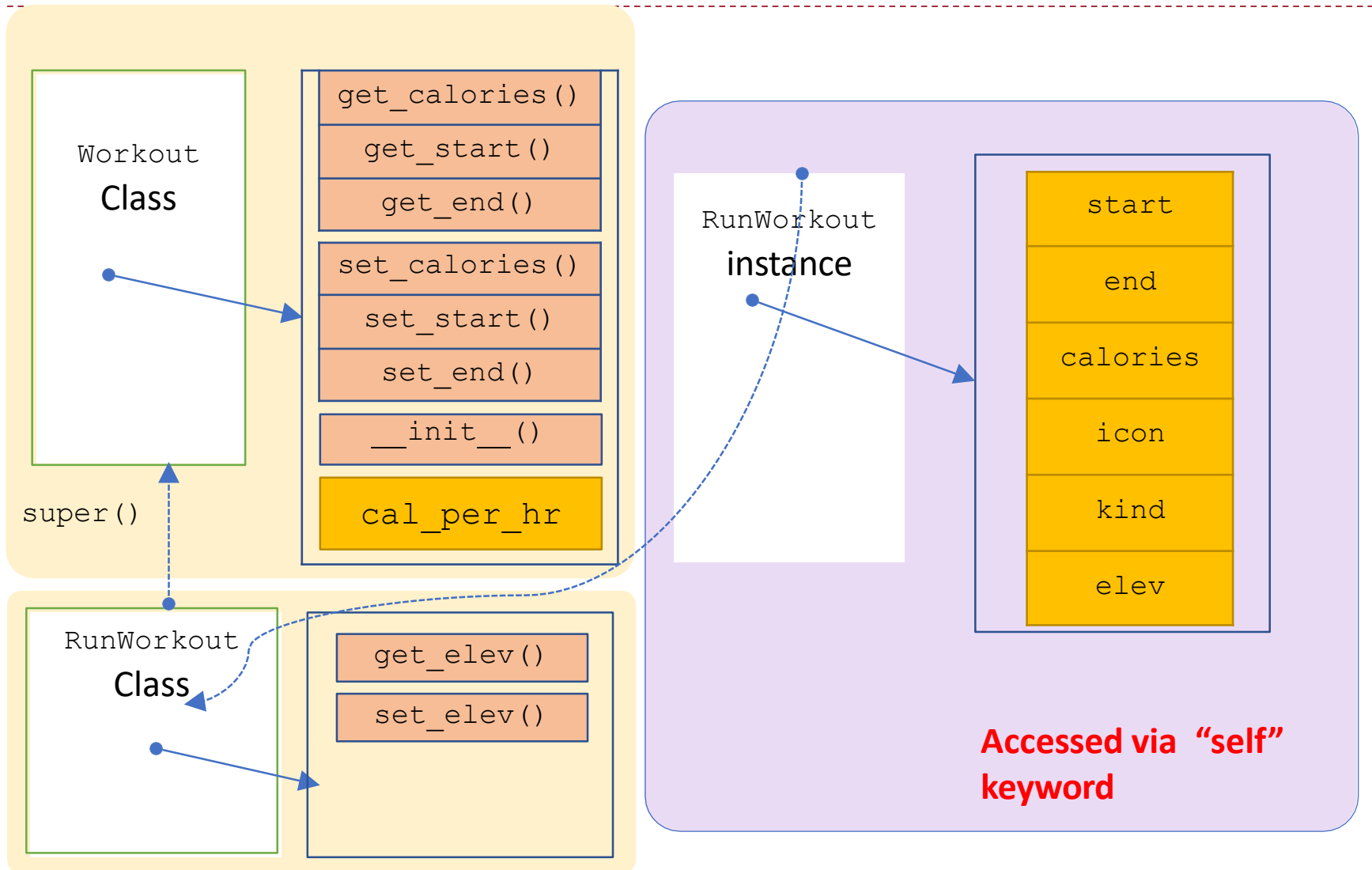
Add new
functionality

Initialize the parent class
(Workout)

Can also call
Workout.__init__(self,
start,end,calories)



INHERITANCE REPRESENTATION IN MEMORY



SUBCLASSES REUSE PARENT CODE

► Complex print function shared by all subclasses

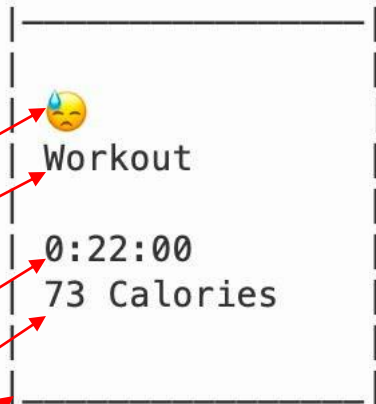
```
class Workout(object)
.....
def __str__(self):

    width = 16
    retstr = f"|{'-'*width}|\n"
    retstr += f"|{' '*width}|\n"
    iconLen = 0
    retstr += f"| {self.icon}{' '*width-3}|\n"
    retstr += f"| {self.kind}{' '*width-len(self.kind)-1}|\n"
    retstr += f"|{' '*width}|\n"
    duration_str = str(self.get_duration())
    retstr += f"| {duration_str}{' '*width-len(duration_str)-1}|\n"
    cal_str = f"{self.get_calories():.0f}"
    retstr += f"| {cal_str} Calories {' '*width-len(cal_str)-11}|\n"

    retstr += f"|{' '*width}|\n"
    retstr += f"|{'_'*width}|\n"

    return retstr
```

outputs



```
|
|  🥵  |
| Workout |
| 0:22:00 |
| 73 Calories |
|         |
| _ _ _ _ _ |
|         |
|         |
```

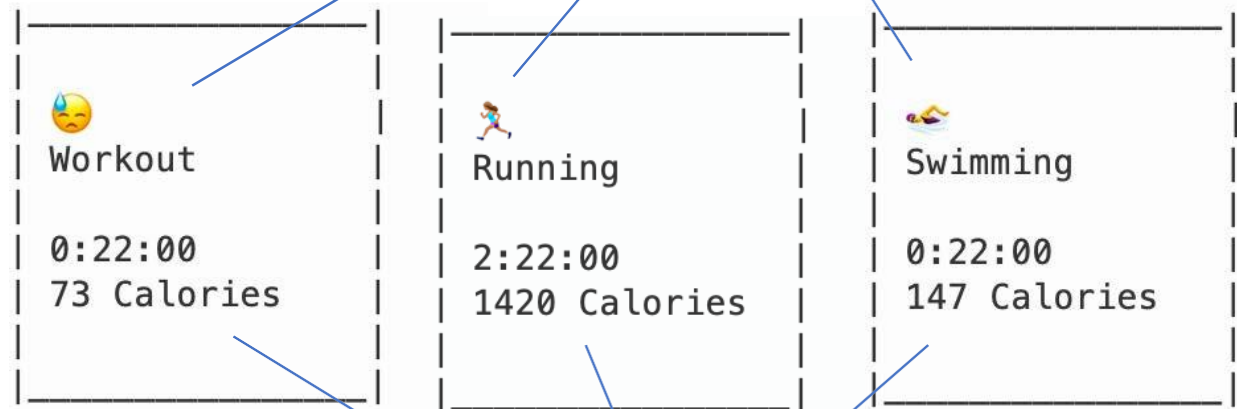
SUBCLASSES REUSE PARENT CODE

```
w=Workout (...)  
rw=RunWorkout (...)  
sw=SwimWorkout (...)
```

```
print(w)  
print(rw)  
print(sw)
```

All Workout subclasses can
use Workout `__str__()`
method!

Workout specific icon
and label



Calories calculated
based on `cal_per_hr`
for each subclass

OVERRIDING SUPERCLASSES

- Overriding superclass – add calorie calculation w/ distance

```
class RunWorkout(Workout):
```

```
    cals_per_km = 100
```

```
    def get_calories(self):
```

```
        if (self.route_gps_points != None):
```

```
            dist = 0
```

```
            lastP = self.routeGpsPoints[0]
```

```
            for p in self.routeGpsPoints[1:]:
```

```
                dist += gpsDistance(lastP,p)
```

```
                lastP = p
```

```
            return dist * RunWorkout.cals_per_km
```

```
        else:
```

```
            return super().get_calories()
```

Add another class var

route_gps_points contains lat/lon pairs of route run

get_calories() overridden since it is defined in both sub and superclass

Iterate through all pairs of GPS points
Summing up their distance

Didn't pass in gps just do whatever the superclass does



OVERRIDING SUPERCLASSES

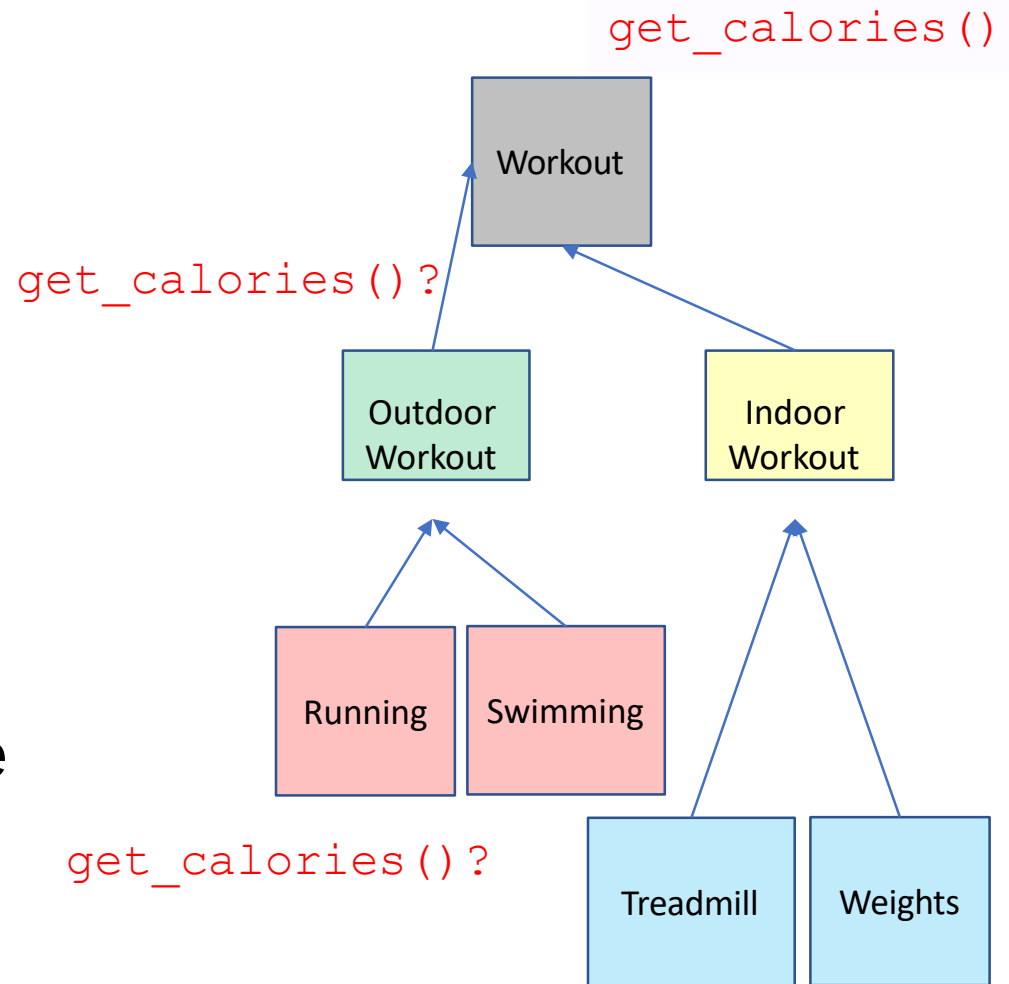
```
class Workout(object)
.....
def __str__(self):
    width = 16
    retstr = f"|{'-'*width}|\n"
    retstr += f"|{' '*width}|\n"
    iconLen = 0
    retstr += f"| {self.icon}{' '*width-3}|\n"
    retstr += f"| {self.kind}{' '*width-len(self.kind)-1}|\n"
    retstr += f"|{' '*width}|\n"
    duration_str = str(self.get_duration())
    retstr += f"| {duration_str}{' '*width-len(duration_str)-1}|\n"
    cal_str = f"{self.get_calories():.0f}"
    retstr += f"| {cal_str} Calories {' '*width-len(cal_str)-11}|\n"
```

- ▶ If invoked by a RunWorkout instance, will call the RunWorkout version of `get_calories()`



WHICH METHOD WILL BE CALLED?

- ▶ For an instance of a class, look for a method name in **current class definition**
- ▶ If not found, look for method name **up the hierarchy** (in parent, then grandparent, and so on)
- ▶ Use first method up the hierarchy that you found with that method name



WHERE CAN I USE AN INSTANCE OF A CLASS?

- ▶ We can use an instance of `RunWorkout` anywhere `Workout` can be used
- ▶ Opposite is not true (cannot use `Workout` anywhere (`RunWorkout` is used))
- ▶ Consider two helper functions

```
def total_calories(workouts):  
    cals = 0  
    for w in workouts:  
        cals += w.get_cals()  
    return cals  
  
def total_elevation(run_workouts):  
    elev = 0  
    for w in run_workouts:  
        elev += w.get_elev()  
    return elev
```



WHERE CAN I USE AN INSTANCE OF A CLASS?

```
def total_calories(workouts):  
    cals = 0  
    for w in workouts:  
        cals += w.get_cals()  
    return cals
```

```
def total_elevation(run_workouts):  
    elev = 0  
    for w in run_workouts:  
        elev += w.get_elev()  
    return elev
```

```
w1 = Workout('9/30/2021 1:35 PM', '9/30/2021 2:05 PM')  
w2 = Workout('9/30/2021 4:35 PM', '9/30/2021 5:05 PM')  
rw1 = RunWorkout('9/30/2021 1:35 PM', '9/30/2021 3:35 PM', 100)  
rw2 = RunWorkout('9/30/2021 1:35 PM', '9/30/2021 3:35 PM', 200)
```

```
total_calories([w1,w2,rw1,rw2])) # (1) # cal = 100+100+400+400  
total_elevation([rw1,rw2]))      # (2) # elev = 100+200  
total_elevation([w1,rw1])        # (3) # err! w1 has no elev method
```



TESTING EQUALITY WITH SUBCLASSES

- ▶ With subclasses, often want to ensure base class is equal, in addition to new properties in the subclass

```
class Workout(object):
```

```
.....
```

```
    def __eq__(self, other):
```

```
        return type(self) == type(other) and \
```

```
            self.startDate == other.startDate and \
```

```
            self.endDate == other.endDate and \
```

```
            self.kind == other.kind and \
```

```
            self.get_calories() == other.get_calories()
```

*And all the other
properties equal too*

Types must be the same

```
class RunWorkout(Workout):
```

```
.....
```

```
    def __eq__(self, other):
```

```
        return super().__eq__(other) and self.elev == other.elev
```

*Workout
properties are
equal*

*And new properties from
RunWorkout are equal*



OBJECT ORIENTED DESIGN: MORE ART THAN SCIENCE

- ▶ OOP is a powerful tool for **modularizing** your code and grouping state and functions together

BUT

- ▶ It's possible to overdo it
 - ▶ New OOP programmers often create elaborate class hierarchies
 - ▶ Not necessarily a good idea
 - ▶ Think about the users of your code
 - ▶ *Will your decomposition make sense to them?*
- ▶ The Internet is full of opinions OOP and “good software design” – you have to develop your own taste through experience!

